FULL-STACK TO AI ENGINEER

PHASE 01 MASTER HANDBOOK (100% COMPLETE)

Phase 01 — JavaScript Master Study Handbook

Comprehensive 16-Topic Engineering Guide: Memory Models, Scope, Closures, Prototypes, Event Loop, Concurrency & Fetch API

Phase 01 — JavaScript Master Study Handbook

> ***Comprehensive Revision Guide:*** *Phase 1 (JavaScript Fundamentals)*
> *Covers Topics 1 to 10 with deep explanations, visual memory models, production bug comparisons, and interview traps.*

Quick Navigation

- Topic 1: Variables (let, const, var), Hoisting & TDZ
- Topic 2: Primitives vs References, Stack vs Heap & Memory
- Topic 3: Operators, Type Coercion, Truthy/Falsy & ?? vs ||
- Topic 4: Loops, Iteration & The Fatal Async forEach Trap
- Topic 5: Functions: Declarations, Expressions, Arrows & this
- Topic 6: Arrays, Objects, Maps, Sets & Immutability
- Topic 7: Destructuring, Spread, Rest & Optional Chaining
- Topic 8: Functional Array Methods (map, filter, reduce, find, some, every)
- Topic 9: Closures, Higher-Order Functions & React Stale Closures
- Topic 10: The this Keyword, Explicit Binding (call, apply, bind) & Prototypes
- Topic 11: Modules — ES Modules (ESM) vs CommonJS (CJS)
- Topic 12: Error Handling — try, catch, finally & Custom Errors
- Topic 13: Promises, Async/Await & Promise Chaining
- Topic 14: The Event Loop, Call Stack, Microtasks vs Macrotasks
- Topic 15: Concurrency — Promise.all, allSettled, race, any
- Topic 16: Fetch API, AbortController, Timeouts & Memory Leaks

Topic 1: Variables (`let` , `const` , `var`), Hoisting & TDZ

1. Mental Model at a Glance

- `let` **(Dry-Erase Board):** You write a value. You can erase and write a new value anytime. Block-scoped `{}`.
- `const` **(Permanent Glue Label):** Glued to one memory box. You cannot pull the label off to point to another box. **However**, you can still change what is *inside* the box if it is an object or array.
- `var` **(The Bad Ghost):** Floats up to the top of the entire function (hoisted as `undefined`), ignoring `if` and `for` blocks.

---

2. Under the Hood: The 2 Execution Passes of V8

```
[ Pass 1: Parsing / Creation Phase ]      [ Pass 2: Execution Phase ]
V8 scans code, registers variable names    V8 executes line-by-line,
and allocates memory locations.            assigns values and runs logic.
```

- `var`**:** Registered and **immediately initialized to** `undefined`.
- `let` **&** `const`**:** Registered in memory, but **UNINITIALIZED**. Placed into the **Temporal Dead Zone (TDZ)**.

---

3. The Temporal Dead Zone (TDZ) & Proof of Hoisting

> ***TDZ Definition:*** *The span of code from the start of a block until the line where a* `let` *or* `const` *is declared. Accessing it here throws a* `ReferenceError`.

Proof that `let` **is Hoisted:**

```
let city = "Surat"; // Global

function test() {
  console.log(city); // 💥 ReferenceError: Cannot access 'city' before initialization
  let city = "Mumbai"; // Local
}
test();
```

- **Why this proves hoisting:** If `let` was NOT hoisted, line 4 would have happily looked outside and printed `"Surat"`. The fact that it crashes proves V8 hoisted `city` locally during Pass 1, shadowed the global variable, and locked it in the TDZ!

---

4. Block Scope vs Function Scope

```
if (true) {
  var leaked = "I escaped!";
  let trapped = "I stay inside!";
}
console.log(leaked);  // "I escaped!" (var completely ignores if blocks!)
console.log(trapped); // 💥 ReferenceError: trapped is not defined
```

---

5. Why `const` Arrays and Objects Can Be Mutated

```
const colors = ["red", "blue"];
colors.push("green"); // ✅ ALLOWED: Memory address of colors didn't change!
colors = ["yellow"];  // ❌ TypeError: Assignment to constant variable!
```

- `const` protects the **memory pointer**, not the heap contents. Use `Object.freeze()` to lock contents.

---

6. The Classic Loop Trap: `var` vs `let`

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3 3 3
```

- **Why with `var`:** There is only **one shared variable `i`** in memory. When the 100ms timers fire, the loop has already completed and `i = 3`. All 3 callbacks look at that same `i`.
- **Why `let` prints `0 1 2`:** JavaScript creates a **brand-new lexical memory binding for every single turn of the loop**. Each callback captures its own private snapshot (`i_0 = 0`, `i_1 = 1`, `i_2 = 2`).

Topic 2: Primitives vs References, Stack vs Heap & Memory

1. Mental Model at a Glance

- **Primitive (A ₹500 Note):** If you give a copy to your friend, they get their own physical note. Tearing their note does not affect yours.
- **Reference Type (A House Address on a Sticky Note):** You copy the address onto your friend's note. You have two notes, but **one house**. If your friend paints the walls red, your house is red too.

---

2. Stack vs Heap Memory Allocation

```
STACK (Fast, fixed-size memory)          HEAP (Dynamic, flexible memory)
-----------------------------------    ----------------------------------------
let age = 25;
let name = "Asha";
let userPtr = 0x00A1  ------------------> 0x00A1: { name: "Ravi", role: "admin" }
let arrPtr  = 0x00B2  ------------------> 0x00B2: [ 10, 20, 30 ]
```

- **Primitives** (`string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`): Live directly on the **Stack**. Copied by value.
- **Objects, Arrays, Functions:** Live on the **Heap**. Variables on the Stack only hold the **memory address pointer**.

---

3. The 3 Legendary Gotchas

Gotcha 1: `typeof null === "object"`

- `null` is a **primitive**, NOT an object.
- This is an unfixed 1995 bug from Brendan Eich's original C-style type-tag system where null pointer `0x00` matched the object tag `000`.
- **Safe Check:** Always check with `val === null`.

Gotcha 2: Floating Point Math (`0.1 + 0.2 !== 0.3`)

- Computers store numbers in binary floating-point (IEEE 754). Binary cannot cleanly represent fractions like $1/10$, creating a tiny rounding error: `0.30000000000000004`.
- **Production Rule for Money: NEVER use floats for currency.** Store money as integer smallest units (cents or paise): `javascript const priceInPaise = 1999; // ₹19.99`

Gotcha 3: `NaN === NaN` is `false`

- `NaN` represents an unrepresentable value. An undefined math value cannot equal another undefined math value.
- Check with: `Number.isNaN(val)`.

---

4. Shallow Copy vs Deep Copy

```
const user = { name: "Asha", address: { city: "Surat" } };

// 1. Shallow Copy (Spread ...): Copies only layer 1
const shallow = { ...user };
shallow.address.city = "Mumbai"; // 💥 Mutates user.address.city too!

// 2. Deep Copy (structuredClone): Clones all nested layers
const deep = structuredClone(user);
deep.address.city = "Delhi"; // ✅ user.address.city remains "Surat"
```

- Avoid `JSON.parse(JSON.stringify(obj))` because it deletes functions, symbols, and `undefined`.

---

5. Why Reference Equality Matters in React

```
{} === {} // false (Two different heap addresses!)
[] === [] // false
```

- React uses shallow reference equality (`Object.is`) to check props. Passing an inline object `<Child config={{ theme: "dark" }} />` forces `Child` to **re-render every single time**, because a new heap address is allocated on every render.

Topic 3: Operators, Type Coercion, Truthy/Falsy & Nullish Coalescing

1. The Only 8 Falsy Values in JavaScript

Everything in JavaScript is truthy **EXCEPT** these 8: 1. `false` 2. `0` 3. `-0` 4. `0n` (BigInt zero) 5. `""` (Empty string) 6. `null` 7. `undefined` 8. `NaN`

> ⚠️ **WARNING**
> In JavaScript, `[]` **(empty array)**, `{}` **(empty object)**, and `"0"` **(string with zero)** are **TRUTHY**! Checking if an array has items must always be: `if (arr.length === 0)`.

---

2. The `||` vs `??` Production Bug

Operator	When does it trigger fallback?	Behavior on `0` and `""`
`||` **(Logical OR)**	On **ANY falsy value** (`0` , `""` , `false` , `null` , `undefined`)	**Overwrites them** (Causes bugs!)
`??` **(Nullish Coalescing)**	**ONLY on** `null` **and** `undefined`	**Preserves them** (Safe for data)

```
const speed = 0; // 0 km/h is a valid speed measurement!

const bug = speed || 50;  // 50 (Bug: treated 0 as missing data!)
const safe = speed ?? 50; // 0  (Correct: preserved valid 0!)
```

---

3. Loose (`==`) vs Strict (`===`) Equality

- `===` **(Strict):** No type coercion. If types differ, returns `false` immediately.
- `==` **(Loose):** Converts types using Abstract Equality. When comparing string/boolean with numbers, **it converts both sides to numbers**:
- `"0" == false` $\rightarrow$ `0 == 0` $\rightarrow$ `true` !
- `0 == ""` $\rightarrow$ `0 == 0` $\rightarrow$ `true` !
- `[] == false` $\rightarrow$ `"" == false` $\rightarrow$ `0 == 0` $\rightarrow$ `true` !
- **Rule:** Never use `==` . Always use `===` .

Topic 4: Loops, Iteration & The Fatal Async `forEach` Trap

1. `for...of` vs `for...in`

```
const fruits = ["apple", "banana", "cherry"];

// for...of gives VALUES:
for (const item of fruits) console.log(item); // "apple", "banana", "cherry"

// for...in gives KEYS / INDICES:
for (const idx in fruits) console.log(idx); // "0", "1", "2" (As strings!)
```

- `for...of` : For iterating over values of iterables (Arrays, Maps, Sets, Strings).
- `for...in` : For inspecting object keys. **Never use on arrays** (returns string indices and iterates prototype properties).

2. Iterating Objects Cleanly

```
const user = { name: "Asha", role: "admin" };

Object.keys(user);   // ["name", "role"]
Object.values(user); // ["Asha", "admin"]

// Modern key-value loop:
for (const [key, val] of Object.entries(user)) {
  console.log(`${key}: ${val}`);
}
```

3. The Fatal `forEach` with `async/await` Bug

```
async function process() {
  console.log("Start");

  [1, 2, 3].forEach(async (num) => {
    await wait(100);
    console.log("Num:", num);
  });

  console.log("End");
}
process();
// Output: Start -> End -> Num: 1 -> Num: 2 -> Num: 3
```

- **Why `"End"` prints before the numbers:** `forEach` is a synchronous loop written in 2009. Internally, it invokes your callback and **throws away the returned Promise**. It never `await`s the callback!

The 2 Production Fixes:

1. **Sequential Waiting (One-by-one):** Use `for...of` (the loop itself honors `await`): `javascript for (const num of [1, 2, 3]) { await wait(100); console.log("Num:", num); } console.log("End"); // Waits for all numbers!`
2. **Parallel / Concurrent Execution (Fastest):** Use `Promise.all`: `javascript await Promise.all([1, 2, 3].map(async (num) => { await wait(100); console.log("Num:", num); })); console.log("End"); // Runs all 3 simultaneously, waits for all!`

Topic 5: Functions: Declarations, Expressions, Arrows & `this`

1. Function Comparison Matrix

Feature	Function Declaration	Function Expression	Arrow Function
Syntax	`function foo() {}`	`const foo = function() {}`	`const foo = () => {}`
Hoisting	**Fully hoisted with body**	Hoisted in TDZ	Hoisted in TDZ
Own `this`?	✅ Yes (depends on caller)	✅ Yes (depends on caller)	❌ **No (lexically inherited)**
Constructor `new`?	✅ Yes	✅ Yes	❌ **No (`TypeError`)**
`arguments`?	✅ Yes	✅ Yes	❌ **No (use `...args`)**

---

2. The 3 Golden Rules of Arrow Functions

1. **No own `this`:** Inherits `this` from the enclosing scope.
2. ❌ **Never use arrow functions for object methods:** `javascript const user = { name: "Asha", greet: () => `Hi ${this.name}` // 'this' points to window/undefined! };`
3. ✅ **Always use arrow functions for internal callbacks:** `javascript numbers.map((n) => n * this.factor); // Borrows outer method's this!`
4. **Cannot be called with `new`:** Arrow functions lack internal `[[Construct]]` methods.
5. **Returning an Object Literal Trap:** Wrap objects in parentheses: `javascript const makeUser = (name) => ({ name }); // Wrap in ()`

Topic 6: Arrays, Objects, Maps, Sets & Immutability

1. The React Immutability Rule

Mutating (Destroys Original) ❌ BANNED IN REACT	Immutable (Returns New Copy) ✅ SAFE IN REACT
`push()`, `pop()`	`concat()`
`shift()`, `unshift()`	`slice()`
`splice()`	`map()`, `filter()`
`sort()`, `reverse()`	`toSorted()`, `toReversed()` (ES2023)

- **Why mutating state breaks React:** React checks `Object.is(prevState, nextState)`. If you do `arr.push(40)` and `setArr(arr)`, the memory address is identical, so React **skips re-rendering completely**.

---

2. The Infamous `.sort()` Alphabetical Bug

```
[10, 5, 40, 25, 100, 1].sort();
// Output: [1, 10, 100, 25, 40, 5]
```

- **Why:** By default, `.sort()` converts all elements to strings and compares their UTF-16 character codes (`"100"` comes before `"25"`).
- **Fix:** Always provide a comparator: `numbers.sort((a, b) => a - b)`.

---

3. `Set` vs `Map`

- `Set` **(Deduplication):** Stores only unique values: `javascript const unique = [...new Set([1, 1, 2, 3])]; // [1, 2, 3]`
- `Map` **(Advanced Key-Value):** Plain objects only allow strings/symbols as keys. `Map` allows **objects, functions, or any type as keys**, maintains insertion order, and has a built-in `.size` property.

Topic 7: Destructuring, Spread, Rest & Optional Chaining

1. Destructuring: Renaming & Defaults

```
const user = { id: 101, username: "asha_dev" };

// Renaming (creates 'handle', does NOT create 'username'):
const { username: handle } = user;

// Defaults (triggers ONLY if value is undefined):
const { role = "guest" } = user;
```

---

2. Spread (`...`) vs Rest (`...`)

- **Spread (** `...` **):** Unpacks / expands items. Used on the **right side** of `=`: `javascript const copy = [...arr]; Math.max(...numbers);`

- **Rest (`...`):** Gathers / collects items into an array. Used on the **left side** of `=` or function parameter lists: `javascript const [first, ...remaining] = [10, 20, 30, 40]; function logAll(...args) {}`

---

3. Optional Chaining (`?.`)

- Evaluates to `undefined` if the operand before `?.` is `null` or `undefined`, preventing crashes: `javascript const city = order?.customer?.address?.city; props.onClick?.(); // Safe function invocation`

Topic 8: Functional Array Methods

```
+-----------+-------------------------+-----------------------------------------+
|  METHOD   |       INPUT -> OUTPUT   |              MENTAL MODEL               |
+-----------+-------------------------+-----------------------------------------+
| .map()    |  N items -> N items     | Transform every single item 1-to-1      |
| .filter() |  N items -> <= N items  | Keep only items matching a condition    |
| .find()   |  N items -> 1 item      | Return FIRST match and STOP immediately |
| .some()   |  N items -> boolean     | Is AT LEAST ONE item matching?          |
| .every()  |  N items -> boolean     | Are ALL items matching?                 |
| .reduce() |  N items -> Single val  | Fold array into sum, object, or map     |
+-----------+-------------------------+-----------------------------------------+
```

Senior Performance: `.find()` vs `.filter()[0]`

- `.filter()[0]` checks every single element in the array and creates a new array in memory.
- `.find()` **stops immediately** on the first match (huge performance boost on large datasets).

Mastering `.reduce()`

```
// 1. Net Balance Calculation:
const balance = transactions.reduce((acc, t) => {
  return t.type === "credit" ? acc + t.amount : acc - t.amount;
}, 0);

// 2. Grouping Objects by Status:
const grouped = orders.reduce((acc, order) => {
  (acc[order.status] ??= []).push(order);
  return acc;
}, {});
```

> ***Warning:*** *Always provide an initial value (`, 0` or `, {}`). Calling `.reduce()` on an empty array without an initial value crashes with `TypeError`.*

Topic 9: Closures, Higher-Order Functions & React Stale Closures

1. The Backpack Model of Closures

- A **Closure** is a function bundled together with references to its surrounding lexical environment.
- When a function is born, it packs all variables in its scope into an invisible "backpack". Even when the parent function has finished executing, the inner function retains its backpack on the **Heap**.

---

2. Real-World Use Case: `debounce()`

```
function debounce(fn, delayMs) {
  let timerId; // Stored in the closure backpack!

  return function(...args) {
    clearTimeout(timerId); // Reset previous timer
    timerId = setTimeout(() => fn(...args), delayMs);
  };
}
```

---

3. The React "Stale Closure" Bug

```
useEffect(() => {
  const timer = setInterval(() => {
    // 💥 Stale closure: captures count = 0 from mount render!
    setCount(count + 1); // Permanently stuck computing 0 + 1 = 1!
  }, 1000);
  return () => clearInterval(timer);
}, []); // Empty dependency array

// ✅ FIX: Use functional state updater:
setCount((prev) => prev + 1);
```

Topic 10: The `this` Keyword, Explicit Binding & Prototypes

1. The 4 Rules Determining `this`

1. **Default Binding:** Standalone call `fn()` $\rightarrow$ `undefined` (in strict mode / ESM) or `window`.
2. **Implicit Binding:** Object method `obj.fn()` $\rightarrow$ `this` is `obj` (the object before the dot).

3. **Explicit Binding:** `.call(ctx)`, `.apply(ctx)` $\rightarrow$ `this` is explicitly set to `ctx`.
4. **New Binding:** `new Fn()` $\rightarrow$ `this` is a brand-new instance object `{}`.
5. **Arrow Functions: No own** `this` (lexically borrowed from enclosing scope).

---

2. `.call()` vs `.apply()` vs `.bind()`

- `.call(ctx, a, b)`: Executes immediately with comma-separated arguments.
- `.apply(ctx, [a, b])`: Executes immediately with an **Array** of arguments.
- `.bind(ctx)`: Does NOT execute immediately; returns a new function with `this` **permanently locked**.

> ***The Permanent*** `.bind()` ***Lock:*** *Once a function is bound with* `.bind(objA)`*, calling* `.call(objB)` *on it later will* ***STILL execute with*** `objA` *! It can never be re-bound.*

---

3. Prototypes Under the Hood

- JavaScript `class` syntax is **syntactic sugar over prototype chains**.
- Methods defined in a class live on `Class.prototype`, not on the individual object instances. 10,000 instances share **one single function in memory**.
- Private fields use the `#` prefix (`#password`) and are enforced natively by the V8 engine.

Topic 11: Modules — ES Modules (ESM) vs CommonJS (CJS)

1. Comparison Matrix

Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax	`require()`, `module.exports`	`import`, `export`
Loading	Dynamic, synchronous, runtime	Static, parsed before execution
Tree-Shaking	❌ Not possible	✅ Built-in (removes unused code)
Top-Level `await`	❌ No	✅ Native support
`__dirname`	✅ Global	❌ Use `import.meta.dirname` (Node 20+)
Top-Level `this`	`module.exports` (`{}`)	`undefined`

2. Best Practices

- **Use Named Exports:** Prefer `export const add = ...` over `export default` for safer IDE refactoring, auto-complete, and better tree-shaking.
- **Avoid Circular Imports:** If Module A imports B and B imports A, one receives an uninitialized empty object, causing runtime `TypeError: Cannot read properties of undefined`. Extract shared code into a 3rd file.

Topic 12: Error Handling — `try`, `catch`, `finally` & Custom Errors

1. The Anatomy of an Error

- **Never throw a plain string:** `throw "error"` discards the **Stack Trace** (file, line number, call history).
- **Always throw an `Error` object:** `javascript throw new Error("Invalid request", { cause: originalError }); // ES2022 error chaining`

2. The 2 Critical Rules of `finally`

1. **`finally` ALWAYS runs:** Even if `try` or `catch` has an explicit `return` or `throw`, `finally` executes before leaving the function.
2. **The `finally` Return Trap:** If you `return` inside `finally`, it **overwrites and discards** any return value or error from `try` or `catch`. Never return from inside `finally`.

3. Custom Error Classes (Production Pattern)

```
class ApiError extends Error {
  constructor(statusCode, message) {
    super(message);
    this.name = "ApiError";
    this.statusCode = statusCode;
  }
}
throw new ApiError(404, "User not found");
```

4. The Async Error Trap

`try/catch` **cannot catch errors from un-awaited Promises**. If you omit `await`, the `try` block exits in 0.001ms, and the subsequent rejection becomes an **Unhandled Promise Rejection**. Always `await` promises inside `try/catch`.

Topic 13: Promises, Async/Await & Promise Chaining

1. Definition & Mental Model

- A **Promise** is a JavaScript object representing the **eventual completion (or failure)** of an asynchronous operation and its resulting value.
- It is a strict finite state machine with **3 mutually exclusive states**:
- `pending`: Initial state. Operation is still running.
- `fulfilled`: Operation completed successfully with a resolved `value`.
- `rejected`: Operation failed with a `reason` (an Error object).
- **The Immutability Rule:** Once a Promise transitions to `fulfilled` or `rejected`, it is **settled** and permanently frozen. Calling `resolve()` or `reject()` a second time does nothing.

2. Core Real-World Use Cases

1. **Network I/O:** Calling REST APIs, GraphQL endpoints, Webhooks, or LLM endpoints (`fetch`, Axios).
2. **Database Operations:** Querying Postgres, MongoDB, or Prisma queries (`await prisma.user.findUnique()`).
3. **File System & Hardware I/O:** Reading/writing files using `node:fs/promises`.
4. **Time & Flow Control:** Promisified timeouts (`delay(1000)`), retry loops, and rate limiters.

3. Promise Chaining Under the Hood

- Every `.then()` and `.catch()` call returns a **brand new Promise**.
- If you return a primitive/object from `.then()`, the next `.then()` receives that value.
- If you return a **Promise** from inside `.then()`, JavaScript automatically **flattens (unwraps)** it before passing the value to the next `.then()`.
- **Trap: Forgetting to Return:** Omitting `return` inside a `.then()` passes `undefined` to the next step in the chain.

4. `async` / `await` Syntax

- `async` functions **always return a Promise** (even if you return a plain number or string, V8 wraps it in `Promise.resolve(...)`).
- `await` pauses the execution of the *surrounding async function* without blocking the main browser/Node thread. It unwraps the fulfilled value or throws the rejection as an error.

5. Critical Production Traps & Mechanical Rules

1. **Synchronous Executor Rule:** The function passed into `new Promise((resolve, reject) => { ... })` runs **IMMEDIATELY and SYNCHRONOUSLY** on the call stack. Only `.then()` callbacks are deferred to the Microtask Queue.
2. **`async` Synchronous Execution Rule:** An `async` function runs **synchronously** from line 1 until the exact moment it hits an `await` expression. Only lines *after* the `await` are suspended and deferred.
3. **The `.catch()` Healing Mechanism:** If `.catch()` handles an error without throwing a new error, it returns a **fulfilled Promise**. Subsequent `.then()` blocks downstream will continue executing.

4. **The Sequential Waterfall Trap:** ```javascript // ❌ SLOW: 2000ms total (waterfall) const user = await getUser(); // 1000ms const posts = await getPosts(); // 1000ms

// ✅ FAST: 1000ms total (parallel execution) const [user, posts] = await Promise.all([getUser(), getPosts()]); `` 5. **The `forEach` + `async` `Trap:**` array.forEach(async (item) => { await save(item); }); `**does NOT wait**.` `forEach` `fires all callbacks synchronously and finishes immediately. Use` for...of `for sequential execution or` Promise.all(array.map(...))` for concurrent execution.

Topic 14: The Event Loop, Call Stack, Microtasks vs Macrotasks

1. The Core Architecture & Mental Model

- **JavaScript is Single-Threaded:** It has **one Call Stack** and executes one line of code at any single instant.
- **Non-Blocking Magic:** I/O, timers, and HTTP requests are offloaded to **Web APIs (in Browser)** or **libuv (in Node.js)** C++ threads. When finished, callbacks are placed into queues.
- **The Airport Runway Mental Model:**
- **The Call Stack (Runway):** Only one plane can take off or land at a time.
- **Microtask Queue (VIP Emergency Lane):** Cleared completely to `0` whenever the Call Stack empties.
- **Macrotask Queue (Standard Taxiway):** Cleared **one task at a time**, after which the controller immediately checks for VIP microtasks again.

2. The Two Queues Compared

Feature	**Microtask Queue (VIP)**	**Macrotask / Task Queue (Standard)**
Sources	`Promise.then/catch/finally`, `await` resumes, `queueMicrotask()`, `process.nextTick`	`setTimeout()`, `setInterval()`, `setImmediate()` (Node), DOM events
Execution Policy	**Drained completely** until `0` remain before next step	**Executed ONE by ONE** per event loop cycle
Priority	Highest	Normal
Risk	Infinite recursion causes **Event Loop Starvation** (freezes UI/Node)	Yields control after every single task

3. The Inviolable 4-Step Event Loop Tick

```
[ Step 1: Run Call Stack ]
       | (Call stack reaches 0)
       ▼
[ Step 2: DRAIN ALL Microtasks ] ◄── Loop here until 0 microtasks left!
       | (Microtask queue is empty)
       ▼
[ Step 3: Render Phase ] ──────────── (Browser only: Layout, Paint if needed)
       |
       ▼
[ Step 4: Run EXACTLY 1 Macrotask ]
       |
       └──► Jump back to Step 2 immediately!
```

4. Critical Interview Traps

1. **The Interleaving Trap (Why `C` runs before `D`):** If a Macrotask schedules a new Microtask, that new Microtask **runs immediately after the current Macrotask finishes**, jumping ahead of any other Macrotasks already waiting in line!
2. **The Microtask Starvation Attack:** `javascript function starve() { queueMicrotask(starve); } starve(); // Locks CPU at 100%, timers/clicks NEVER execute!`

Topic 15: Concurrency — `Promise.all`, `allSettled`, `race`, `any`

1. Simple Definitions & Mental Models (Easy Language)

- **Waterfall (Sequential):** Waiting for one async task to finish before starting the next (slow).
- **Concurrent:** Firing multiple async tasks at the exact same instant (fast).

Method	Plain English Analogy	The Golden Rule
`Promise.all`	**The Bank Heist Crew:** All members must succeed. If one gets caught, the whole heist is blown immediately.	**"All or Nothing"** (Fail-fast).
`Promise.allSettled`	**The School Exam:** Teacher waits for every student's paper, whether they passed or failed.	**"Wait for everybody, no matter what"** (Never throws).
`Promise.race`	**Formula 1 Race:** First car across the finish line wins, whether it wins cleanly or crashes across.	**"First to settle wins"** (Success or Failure).
`Promise.any`	**Hailing a Taxi:** You ask 3 cab apps simultaneously. First driver to accept wins. You only cry if all 3 apps reject.	**"First SUCCESS wins"** (Ignores errors unless all fail).

2. Deep Mechanics & Code Patterns

A. `Promise.all([p1, p2, p3])`

- Resolves to an array of results with **preserved input order**.
- **Fail-fast:** Rejects immediately upon the first rejection.
- **The "Zombie" Trap:** Unsettled promises in the array **continue running** in the background! They are not cancelled unless linked to an `AbortController`.

B. `Promise.allSettled([p1, p2, p3])`

- Waits for all promises to finish (either fulfilled or rejected).
- Returns an array of status objects:
- `{ status: "fulfilled", value: data }`
- `{ status: "rejected", reason: error }`
- Use case: Multi-widget dashboards, bulk email / push notification processing.

C. `Promise.race([p1, p2, p3])`

- Settles as soon as the **first** promise settles (fulfills OR rejects).
- Essential Production Pattern: **Request Timeout**. `javascript const data = await Promise.race([ fetch('/api/data'), new Promise((_, reject) => setTimeout(() => reject(new Error('Timeout')), 5000)) ]);`

D. `Promise.any([p1, p2, p3])`

- Settles as soon as the **first promise fulfills** (ignores failures).
- If all reject, throws an `AggregateError` containing `error.errors` (array of all failure reasons).
- Use case: Redundant CDNs, multi-region fallback endpoints.

3. Edge Cases & Interview Traps

- **Empty Arrays `[]`:**
- `Promise.all([])` $\rightarrow$ Resolves immediately to `[]`.
- `Promise.allSettled([])` $\rightarrow$ Resolves immediately to `[]`.
- `Promise.any([])` $\rightarrow$ Rejects immediately with `AggregateError`.
- `Promise.race([])` $\rightarrow$ **STAYS PENDING FOREVER!** (Never resolves or rejects; leaks memory).
- **Non-Promise Values:** Automatically wrapped in `Promise.resolve(val)`.

Topic 16: Fetch API, `AbortController`, Timeouts & Memory Leaks

1. Simple Definitions & Mental Models (Easy Language)

- **`fetch()`:** The courier between your browser and server.
- **The Fatal Trap:** The courier **never fails** on `404 Not Found` or `500 Server Error`. As long as the courier reaches the server and brings back a status message, the promise **resolves**!
- **When does `fetch()` reject?** ONLY on physical network failures (Wi-Fi dropped, offline, invalid domain, DNS lookup failed).
- **Rule:** Always check `if (!response.ok)` (`status` between 200–299).
- **`AbortController`:** The "Cancel" button for in-flight requests.
- Stops wasting network bandwidth and prevents race conditions and memory leaks.
- **The Juice Box Stream Analogy:**
- HTTP response bodies are **one-way data streams**.
- Calling `await response.json()` or `await response.text()` drains the carton completely.
- Calling `response.json()` a second time throws `TypeError: Failed to execute 'json' on 'Response': body stream already read`. (Use `response.clone()` if you truly need multiple reads).

2. Under the Hood Mechanics & Code

A. Checking `response.ok` Properly

```
const response = await fetch('/api/user/10');
if (!response.ok) {
  throw new Error(`HTTP Error ${response.status}: ${response.statusText}`);
}
const data = await response.json();
```

B. `AbortController` Pattern

```
const controller = new AbortController();

fetch('/api/heavy-data', { signal: controller.signal })
  .then(res => res.json())
  .catch(err => {
    if (err.name === 'AbortError') {
      console.log('Request aborted cleanly!');
    } else {
      console.error('Real network failure:', err);
    }
  });

// Cancel the request at any time:
controller.abort();
```

C. Modern Native Timeout with `AbortSignal.timeout(ms)`

```
try {
  // Automatically cancels after 4000ms:
  const res = await fetch('/api/data', { signal: AbortSignal.timeout(4000) });
  const data = await res.json();
} catch (err) {
  if (err.name === 'TimeoutError') {
    console.error('Request timed out!');
  }
}
```

3. Production Bugs Prevented

1. **Typeahead / Search Bar Race Conditions:** Typing "cat", then "catering". Aborting the previous request ensures slow stale responses never overwrite newer search results.
2. **React Memory Leaks / Zombie Callbacks:** Aborting in the `useEffect` cleanup prevents setting state on unmounted components.